POO

Sabloane de proiectare

Cuprins

- Sabloane de proiectare (software design patterns)
- singleton
- Composite
- Visitor
- Object Factory

Sabloane de proiectare (Design Patterns)

- intai aplicate in proiectare urbanistica:
  C. Alexander. A Pattern Language. 1977

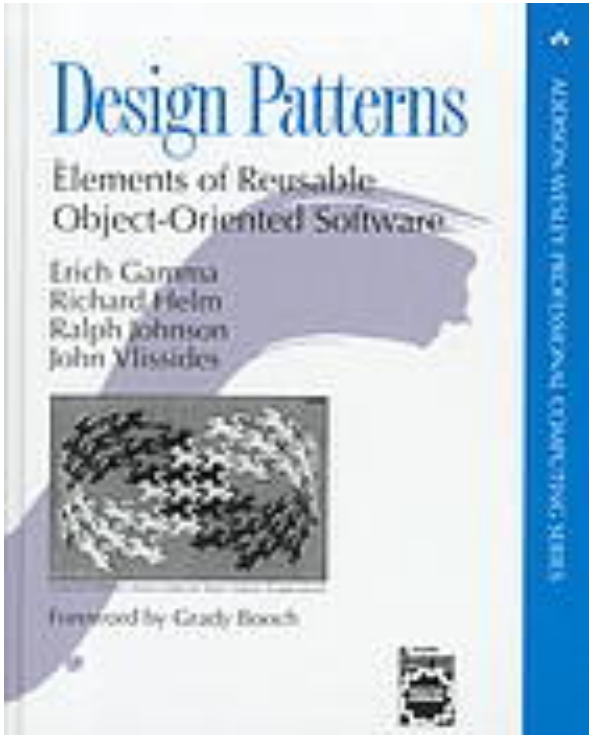
- definitia originala a lui Alexander: *"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice"*

- prima contributie in software: 1987, Kent Beck (creatorul lui Extreme Programming) & Ward Cunningham (a scris primul wicki)

Aplicarea sabloanelor in POO

- contributia majora: Design Patterns:
  Gamma et al. Elements of Reusable Object-Oriented Software was published, 1994
  - cunoscuta ca **GoF** (Gang of Four)
  - pot fi aplicate la nivel de clasa sau obiect

- GoF include 23 de sabloane



Formatul (template) complet al unui sablon

- nume si clasificare
- intentie
- cunoscut de asemenea ca
- motivatie
- aplicabilitate
- structura
- participanti
- colaborari
- consecinte
- implementare
- cod
- utilizari cunoscute
- sabloane cu care are legatura

Clasificarea sabloanelor GoF

- creationale
  - utilizate pentru crearea obiectelor
- structurale
  - utilizate pentru a defini structura (compunerea) claselor sau obiectelor
- comportamentale
  - descrie modul in care clasele si obiectele interactioneaza si isi distribuie responsabilitatile

POO

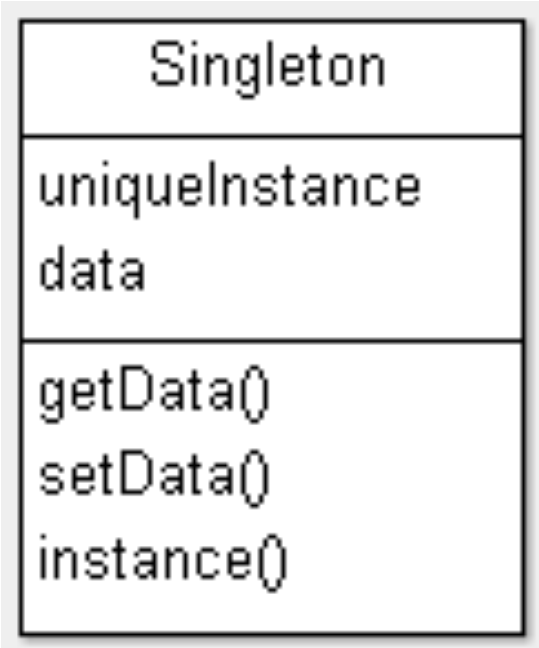
Singleton
(prezentare bazata pe GoF)

Singleton

- Clasificare
  - creational
- Intentia
  - proiectarea unei clase cu un singur obiect (o singura instanta)
- Motivatie
  - intr-un sistem de operare:
    - exista un sistem de fisiere
    - exista un singur manager de ferestre
  - intr-un sit web: exista un singur manager de pagini web
- Aplicabilitate
  - cand trebuie sa existe exact o instanta
  - clientii clasei trebuie sa aiba acces la instanta din orice punct bine definit

Singleton

- structura



- participant: Singleton
- collaborations: clients of the class

Clase cu o singura instanta (Singleton)

- Consecinte
  - acces controlat la instanta unica
  - reducerea spatiului de nume (eliminarea variab. globale)
  - permite rafinarea operatiilor si reprezentarii
  - permite un numar fix de instante
    - Doubleton
    - Tripleton
    - ...
  - mai flexibila decat operatiile la nivel de clasa (statice)
- Implementare

cum?

Clase cu o singura instanta

```
template <typename Data>
class Singleton {
public:
  static Singleton<Data>& instance() {
    return uniqueInstance;
  }
  Data getData() { return data; }
  void setData(Data x) { data = x; }
protected:
  Data data; // campurile care descriu starea instantei
  Singleton() { }
  void operator=(Singleton&);
  Singleton(const Singleton&);
private:
  static Singleton<Data> uniqueInstance;
};
```

manerul cu care se are acces la instanta

metode care opereaza peste instanta unica

constructor

operator atribuire

constructor de copiere

instanta unica

Clase cu o singura instanta

```
template <typename Data>
Singleton<Data> Singleton<Data>::uniqueInstance;

int main() {
  Singleton<int> &s1 = Singleton<int>::instance();
  cout << s1.getData() << endl; // 0
  Singleton<int> &s2 = Singleton<int>::instance();
  s2.setData(9);
  cout << s1.getData() << endl;  // 9
  Singleton<int> &s3 = s2;
  cout << s3.getData() << endl;  // 9
}
```

initializare

refera aceeasi instanta (pe cea unica)

Clase cu o singura instanta

- daca se comenteaza constructorul de copiere, atunci se poate executa urmatorul cod:

```
Singleton<int> s4 = s2;
s4.setData(23);
cout << s4.getData() << endl; // 23
cout << s2.getData() << endl; // 29
```

- operatorul de atribuire se declara privat doar pentru a evita operatii fara sens ca:

```
s4 = s2;
```

- in lipsa accesului la constructori, operatorul de atribuire nu poate fi utilizat pentru doua instante diferite

Demo

C++ 2011

- interzicerea utilizarii unor functii/metode se poate face printr-o declaratie **delete**
- din manual (8.4.3):

"1 A function definition of the form:

attrib-specifier-seqopt decl-specifier-seqopt declarator = **`delete`** ;

is called a **deleted definition**. A function with a deleted definition is also called a deleted function.

2 A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed.

[Note: This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. If a function is overloaded, it is referenced only if the function is selected by overload resolution. — end note]"

C++ 2011

- inhibarea utilizarii constructorului de copiere si operatorului de atribuire

```
class Singleton {
public:
   …
   void operator=(Singleton&) = delete;
   Singleton(const Singleton&) =  delete;
   …
private:
   …
}
```

C++ 2011

- C++ 2011 include si constructorul de mutare

- din manual (12.8):

"3 A non-template constructor for class X is a move constructor if its first parameter is of type

X&&,

`const` X&&,

`volatile` X&&, or

`const volatile` X&&,

and either there are no other parameters or else all other parameters have default arguments (8.3.6).

(continuare pe slide-ul urmator)

C++ 2011

[Example: Y::Y(Y&&) is a move constructor.

```
struct Y {
  Y(const Y&);
  Y(Y&&);
};
extern Y f(int);
Y d(f(1)); // calls Y(Y&&)
Y e = d; // calls Y(const Y&)
```

— end example]”

- Ar trebui ascuns si constructorul de mutare?

C++ 2011:

- Constructorul de mutare nu definit intotdeauna intotdeauna
- Din manual (12.8):

"10 If the definition of a class X does not explicitly declare a move constructor, one will be implicitly declared as defaulted if and only if

— X does not have a user-declared copy constructor,

— X does not have a user-declared copy assignment operator,

— X does not have a user-declared move assignment operator,

— X does not have a user-declared destructor, and

— the move constructor would not be implicitly defined as deleted."

Instanta unica dinamica 1/2

```
template <typename Data>
class Singleton {
public:
  static Singleton* instance() {
    if (uniqueInstance == 0) {
      uniqueInstance = new Singleton();
  }
    return uniqueInstance;
  }
  // metode care opereaza peste instanta
  unica
  Data getData() { return data; }
  void setData(Data x) { data = x; }
```

Dynamic unique instance 2/2

```
protected:
  Data data;
  Singleton()  { }
  // void operator=(Singleton&);
  // Singleton(const Singleton&);

private:
  static Singleton<Data>* uniqueInstance;
};
```

Nu mai e necesar de ascuns
De ce?

Pointer la instanta unica

Demo

```
Singleton<int>* s1 = Singleton<int>::instance();
s1->setData(47);
cout << s1->getData() << endl;  // 47
Singleton<int>* s2 = Singleton<int>::instance();
s2->setData(9);
cout << s1->getData() << endl;  // 9
cout << s2->getData() << endl;  // 9
```

Diferenta dintre pointer si referinta 1/2

- urmatoarele instructiuni se executa, indiferent cum declaram constructorul de copiere si/sau operatorul de atribuire

```
Singleton<int>* s4 = s2;
s4->setData(23);
cout << s4->getData() << endl;
cout << s2->getData() << endl;

s4 = s1;
s4->setData(43);
cout << s4->getData() << endl;
cout << s2->getData() << endl;
```

de ce?

Diferenta dintre pointer si referinta 2/2

- dar urmatoarea instructiune nu se compileaza daca se decomenteaza constructorul de copiere

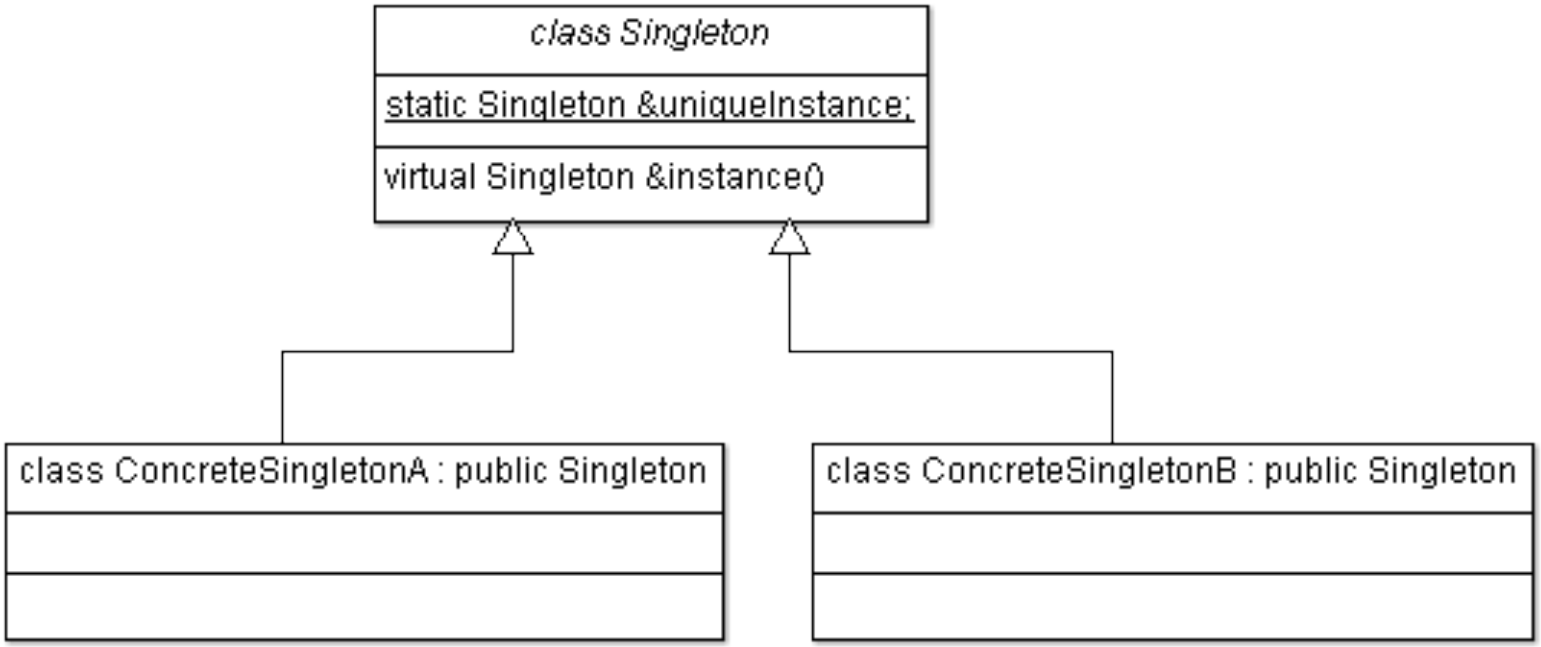
```
Singleton s5 = (*s2);
```

- si urmatoarea instructiune nu se compileaza daca se decomenteaza si operatorul de atribuire

```
s5 = *s1;
```

Demo

Clase singleton derivate



- pot fi probleme la crearea instantelor claselor concrete din ierarhie => repozitoriu de clase Singleton

POO

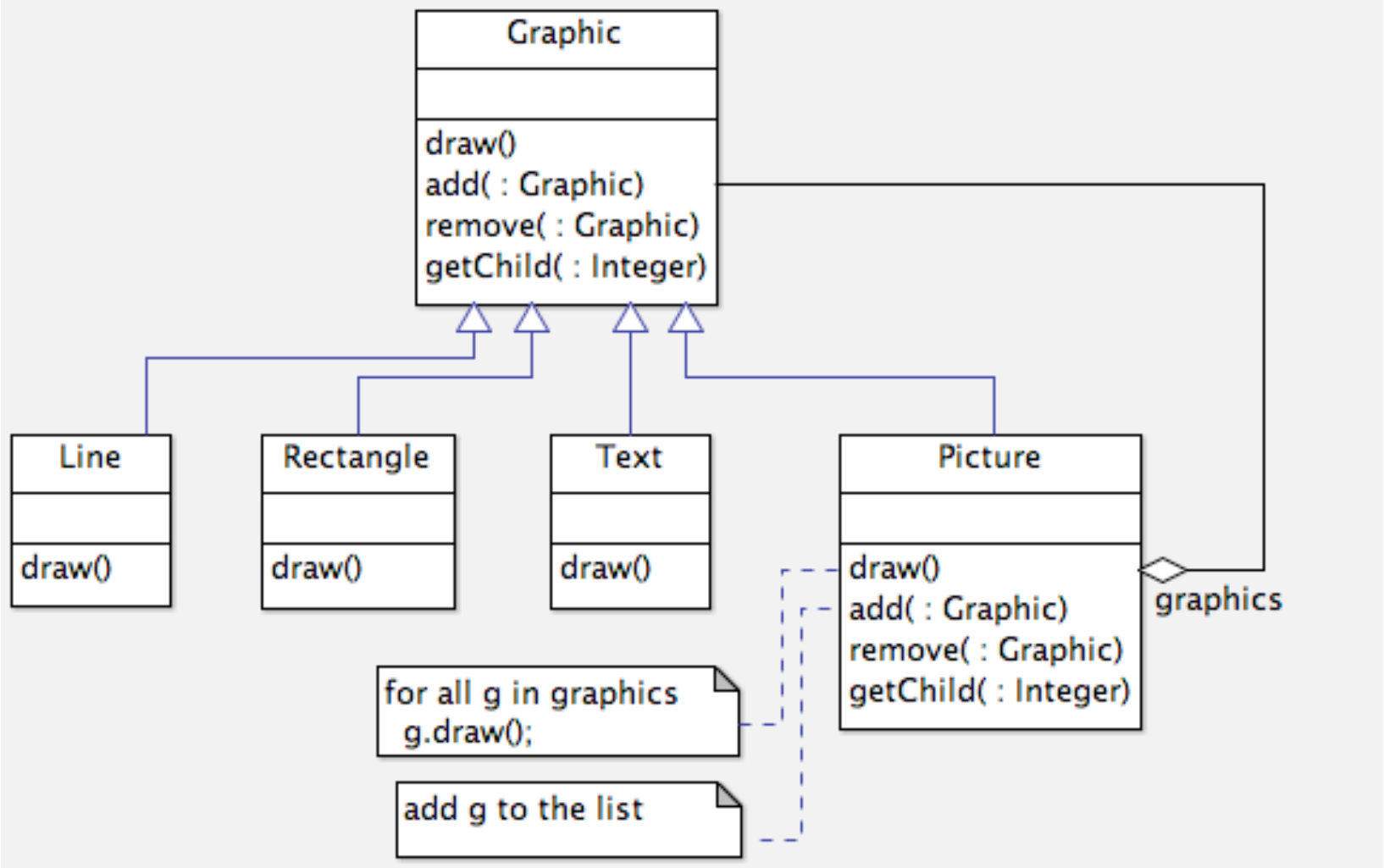
Composite

(prezentare bazata pe GoF)

Composite::intentie

- este un pattern structural
- compune obiectele intr-o structura arborescenta pentru a reprezenta o ierarhie parte-intreg
- lasa clientii (structurii) sa trateze obiectele individuale si compuse intr-un mod uniform

Composite:: motivatie



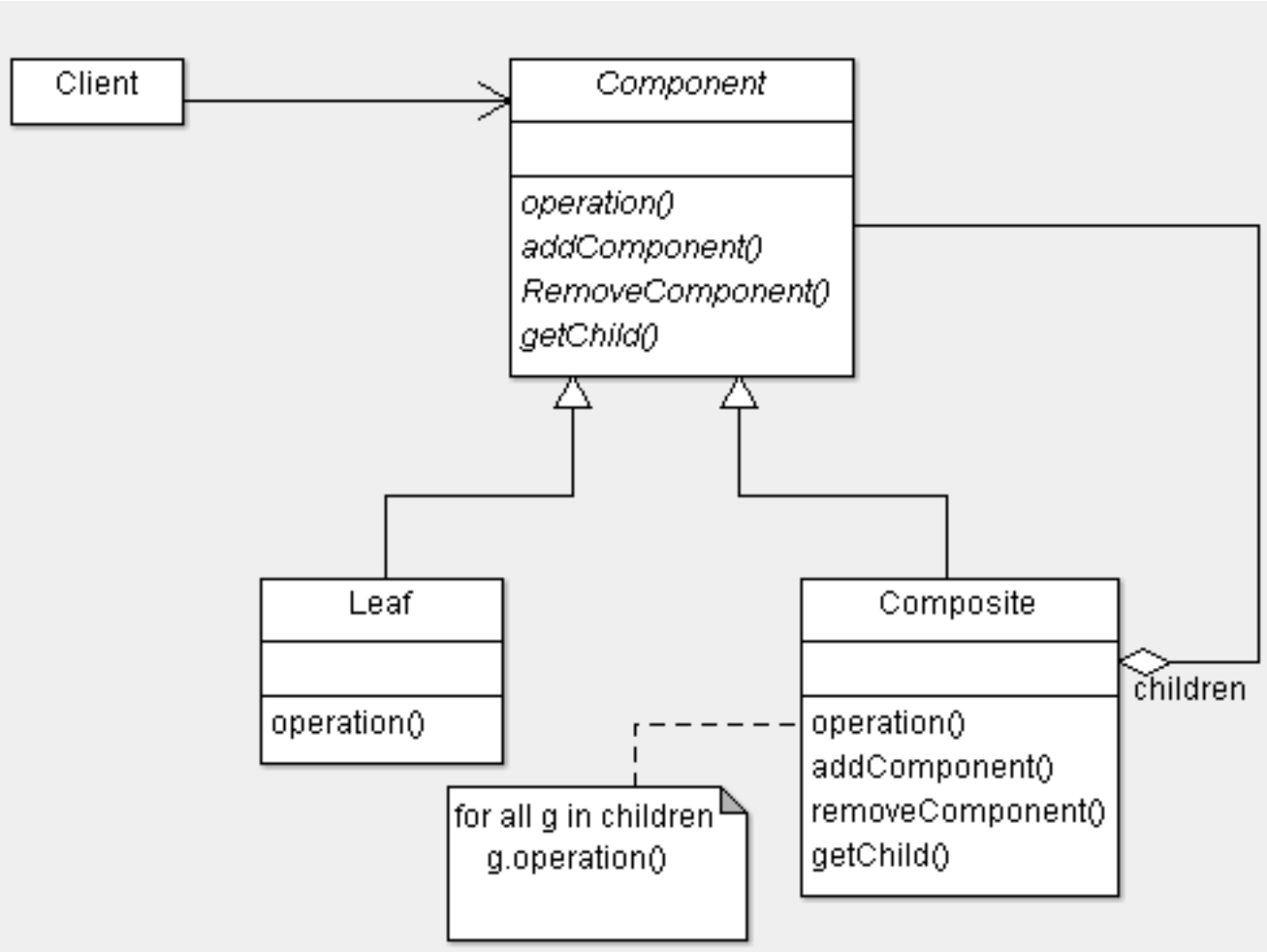
Composite:: caracterul recursiv al str.

- orice (obiect) linie este un obiect grafic
- orice (obiect) dreptunghi este un obiect grafic
- orice (obiect) text este un un obiect grafic
- o pictura formata din mai multe obiecte grafice este un obiect grafic

Composite::aplicabilitate

- pentru a reprezenta ierarhii parte-intreg
- clientii (structurii) sa poata ignora diferentele dintre obiectele individuale si cele compuse
- obiectele structurii posibil sa fie tratate uniform (cu un anumit pret)

Composite::structura



Composite::participanti

- **Component (Graphic)**
  - declara interfata pentru obiectele din compozitie
  - implementeaza comportarea implicita pentru interfata comuna a tuturor claselor
  - declara o interfata pentru accesarea si managementul componentelor-copii
  - (optional) defineste o interfata pentru accesarea componentelor-parinte in structura recursiva
- **Leaf (Rectangle, Line, Text, etc.)**
  - reprezinta obiectele primitive; o frunza nu are copii
  - defineste comportarea obiectelor primitive

Composite::participanti

- **Composite (Picture)**
  - defineste comportarea componentelor cu copii
  - memoreaza componentele-copil
  - implementeaza operatiile relative la copii din interfata *Component*
- **Client**
  - manipuleaza obiectele din compozitie prin intermediul interfetei *Component*

Composite::colaborari

- clientii utilizeaza clasa de interfata *Component* pentru a interactiona cu obiectele din structura
- daca recipientul este o instanta *Leaf*, atunci cererea este rezolvata direct
- daca recipientul este o instanta *Composite,* atunci cererea este transmisa mai departe componentelor-copil; alte operatii aditionale sunt posibile inainte sau dupa transmitere

Composite::consecinte

- defineste o ierarhie de clase constand din obiecte primitive si compuse
- obiectele primitive pot fi compuse in obiecte mai complexe, care la randul lor pot fi compuse in alte obiecte mai complexe samd (recursie)
- ori de cate ori un client asteapta un obiect primitiv, el poate lua de asemnea si un obiect compus
- pentru client e foarte simplu; el trateaza obiectele primitive si compuse in mod uniform
- clientului nu-i pasa daca are de-a face cu un obiect primitiv sau compus (evitarea utilizarii structurilor de tip *switch-case*)

Composite:: consecinte

- este usor de adaugat noi tipuri de componente *Leaf* sau *Composite*; noile subclase functioneaza automat cu structura existenta si codul clientului. Clientul nu schimba nimic.
- face designul foarte general
- dezavantaj: e dificil de restrictionat ce componente pot sa apara intr-un obiect compus (o solutie ar putea fi verificarea in timpul executiei)

Composite::implementare

- *Referinte explicite la* parinte.
  - simplifica traversarea si managementul structurii arborescente
  - permite travesarea bottom-up si stergerea unei componente
  - referinta parinte se pune in clasa *Component*
  - usureaza mentinerea urmatorului invariant: parintele unui copil este un obiect compus si-l are pe acesta ca si copil (metodele add() si remove() sunt scrise o singura data si apoi mostenite)

Composite::implementare

- *Componente partajate*.
  - cateodata este util sa partajam componente
  - … dar daca o componenta are mai mult decat un parinte, atunci managementul devine dificil
  - o solutie posibila: parinti multipli (?)
  - exista alte patternuri care se ocupa de astfel de probleme (Flyweigth)

Composite::implementare

- *Maximizarea interfetei Component*
  - *Component* ar trebui sa implementeze cat mai multe operatii comune (pt *Leaf* si *Composite*)
  - aceste operatii vor descrie comportarea implicita si pot fi rescrise de *Leaf* si *Composite* (sau subclasele lor)
  - totusi aceasta incalca principiul "Single responsibility principle" (o clasa trebuie sa aiba o singura responsabilitate si sa implementeze numai ce are sens pentru subclase" unele operatii pentru *Composite* nu au sens pt. *Leaf* (sau invers)
  - de ex. *getChild()*
    - solutie: comportarea default = nu intoarce niciodata vreun copil

Composite:: implementare

- *Operatiile de management a copiilor (cele mai dificile)*
  - unde le declaram?
  - daca le declaram in *Component*, atunci avem *transparenta* (datorita uniformitatii) dare ne costa la siguranta (safety) deoarece clientii pot incerca op fara sens (ex. eliminarea copiilor unei frunze)
  - daca le declaram in *Composite*, atunci avem *siguranta* dar nu mai avem transparenta (avem interfete diferite pt comp. primitive si compuse)
  - patternul opteaza pentru transparenta

Composite:: implementare

- ce se intampla daca optam pentru siguranta?
- se pierde informatia despre tip si trebuie convertita o instanta *Component* intr-o instanta *Composite*
- cum se poate face?
- o posibila solutie: declara o operatie

  Composite getComposite()*

  in clasa *Component*
- *Component* furnizeaza comportarea implicita intorcand un pointer NULL
- *Composite* rafineaza operatia intorcandu-se pe sine insasi prin itermediul pointerului *this*

Implementarea metodei getComposite()

```
class Composite;
class Component {
public:
  //...
  virtual Composite* getComposite() { return 0; }
};
class Composite : public Component {
public:
  void Add(Component*);
  // ...
  virtual Composite* getComposite() { return this; }
};
class Leaf : public Component {
// ...
};
```

problema de tip “oul sau gaina”

implementarea implicita

implementarea pt. un compus

Exemplu utilizare a metodei getComposite()

```
Composite* aComposite = new Composite;
Leaf* aLeaf = new Leaf;
Component* aComponent;
Composite* test;
cin >> option;
if (option == 1)
  aComponent = aComposite;
else
  aComponent = aLeaf;
if (test = aComponent->getComposite()) {
  test->Add(new Leaf);
}
```

crearea unui obiect compus si a unei frunze

option == 1:
adauga, pentru ca test va fi diferit de zero
option != 1:
NU adauga, pentru ca test va fi zero

Composite:: implementare

- evident, componentele nu sunt tratate uniform
- singura posibilitate de a avea transparenta este includerea operatiile relativ la copii in *Component*
- este imposibil de a implementa *Component:add()* fara a intoarce o exceptie (esec)
- ar fi ok sa nu intoarca nimic?
- ce se poate spune despre *Component:remove()* ?

Composite:: implementare

- *Ar trebui implementata o lista de fii in Component*?
  - ar fi tentant
  - dar …
  - … ar fi irosire de spatiu
- *Ordinea copiilor*
  - sunt aplicatii in care conteaza
  - daca da, atunci accesul si managementul copiilor trebuie facut cu grija
- *Cine sterge componentele?*
  - fara GC, responsabilitatea este a lui *Composite*
  - atentie la componentele partajate
- *Care structura e cea mai potrivita pentru lista copiilor?*

Studiu de caz

Expresii intr-un limbaj de programare

- expresii aritmetice: a + b * 2 - c
- expresii relationale: a + 2 < b * 3
- expresii booleene: a < 3 && (b < 0 || a < b)

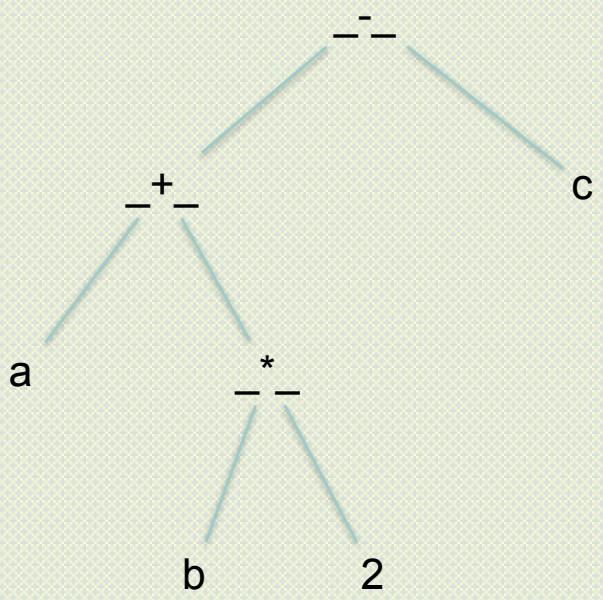
Expresii aritmetice

```
PrimaryExpression ::=
      IntConstant
    | VarName
    | "(" Expression ")"

IntConstant ::=
      Digit+

Digit ::=
     "0" | "1" | ... | "9"

VarName ::=
     "a" | "b" | ... | "z"

MultiplicativeExpression ::=
      PrimaryExpression (("*" | "/" | "%") PrimaryExpression)*

ArithExpression ::=
      MultiplicativeExpression (("+" | "-") MultiplicativeExpression)*

Expression ::= ArithExpression
```

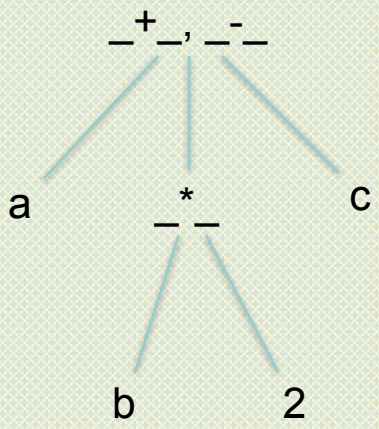
Cracaterul recursiv al definitiei

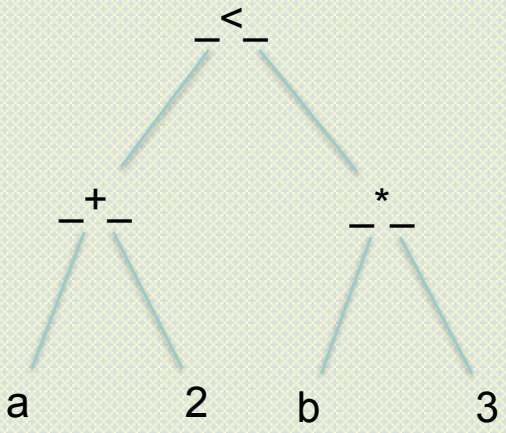
- pe tabla

Reprezentarea expresiilor ca arbori sintactici abstracti (AST)



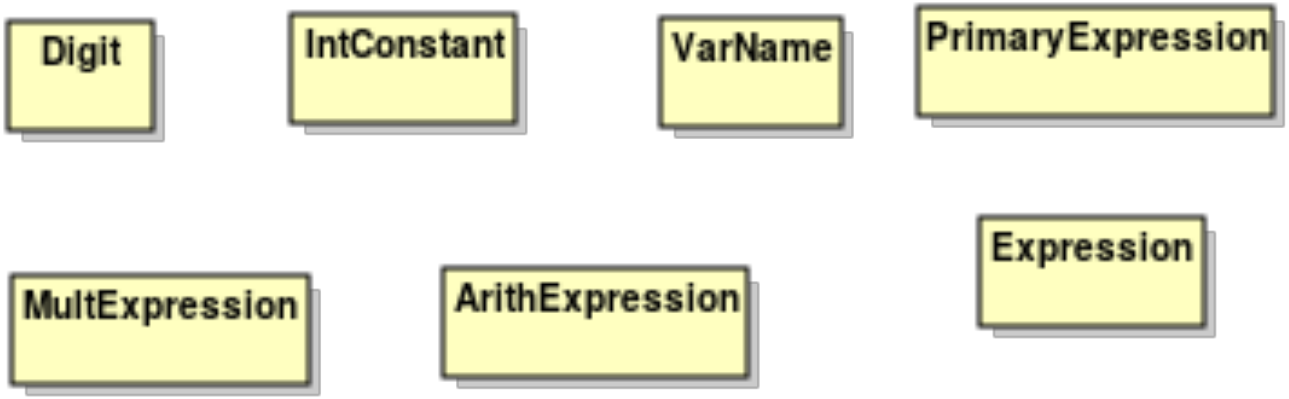
sau





Reprezentarea ASTului cu clase

- fiecare categorie sintactica o clasa



Relatii intre clase

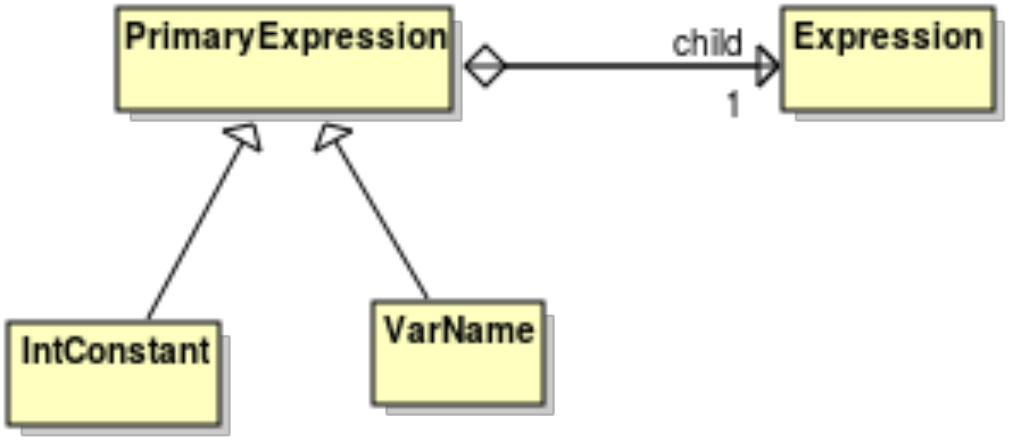
PrimaryExpression ::=
IntConstant
| VarName
| "(" Expression ")"

=

PrimaryExpression ::=
IntConstant
| VarName

+

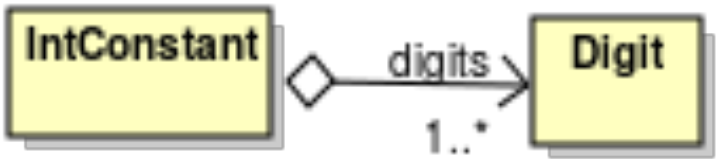
PrimaryExpression ::=
"(" Expression ")"



Relatii intre clase

IntConstant ::=

Digit+



MultiplicativeExpression ::=

PrimaryExpression (("*" | "/" | "%") PrimaryExpression)*

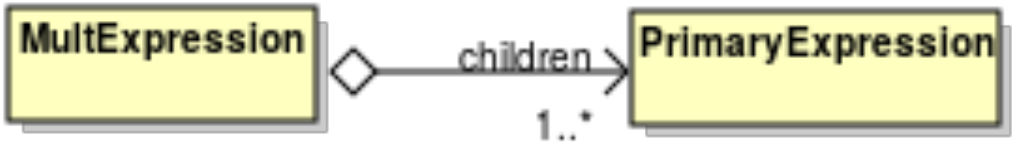
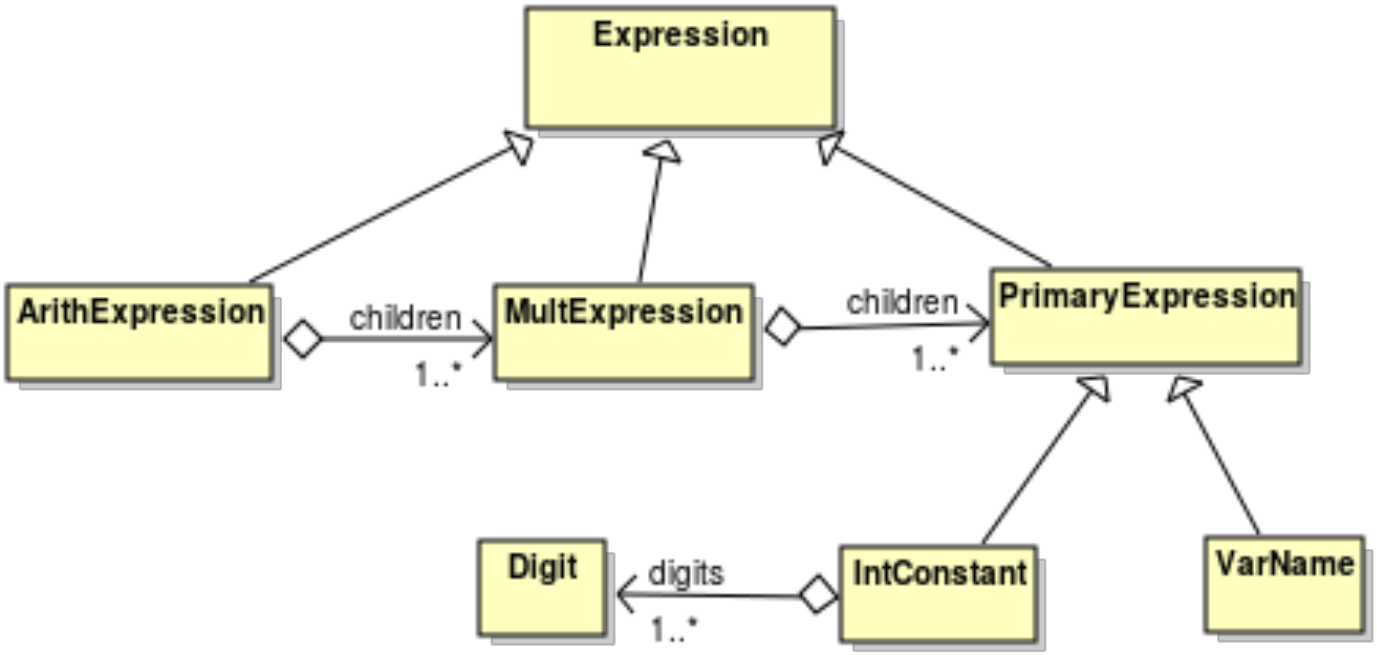
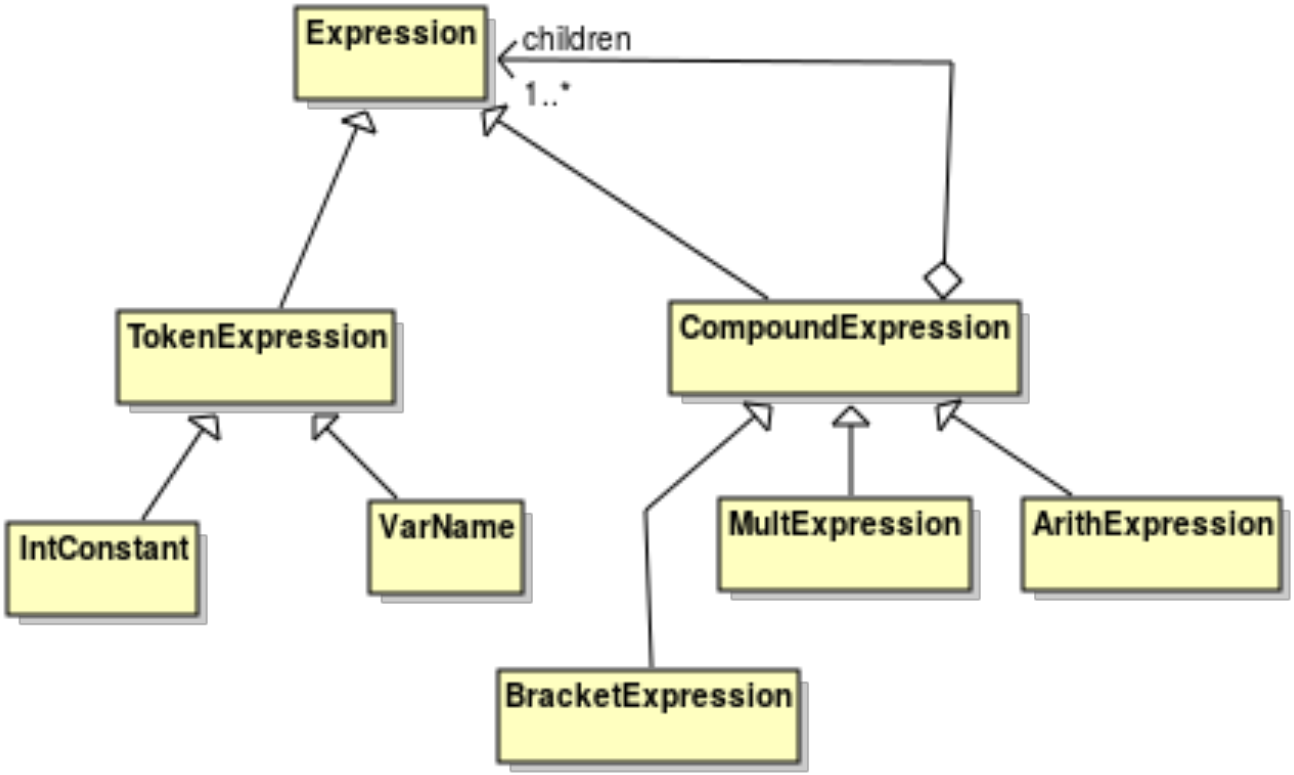


Diagrama completa



Ce principiu nu respecta diagrama de mai sus?

Diagrama urmand modelul Composite



Clasa Expression - C++

```
class Expression {
 public: virtual list<string> getLabel();
 public: virtual void addLabel(string str);
 public: virtual string toString();
 public: virtual list<string> getLabel();
 protected: list<string> label;
};
```

Clasa CompoundExpression

```
class CompoundExpression : public Expression {
 public: void addChild(Expression* pe);
 public: string toString();
 public: list<Expression*> getChildren();
 protected: list<Expression*> children;
};

class ArithExpression : public CompoundExpression {
};

class MultExpression : public CompoundExpression {
};

….
```

Parserul

- descrierea este inclusa intr-o prezentare separata accesibila pe pagina cursului

Demo

```
int main() {
  char str[80];
  std::cout << "Input: ";
  std::cin.getline(str, 80);
  Parser p(str);
  //Parser p(" b + a * 3 / 5");
  std::optional<Expression*> ast = p.expression();
  if (ast.has_value()) {
    std::cout << ast.value()->toString() << "\n"
              << p.getIndex() << "\n";
  }
  return 0;
}
```

C++17: explicatii la curs

Demo

```
$ ./a.out
Input: a + b / 2 + 3
(a + (b / 2) + 3)
13
```

POO

Sablonul

Visitor

(prezentare bazata pe GoF)

Intentie

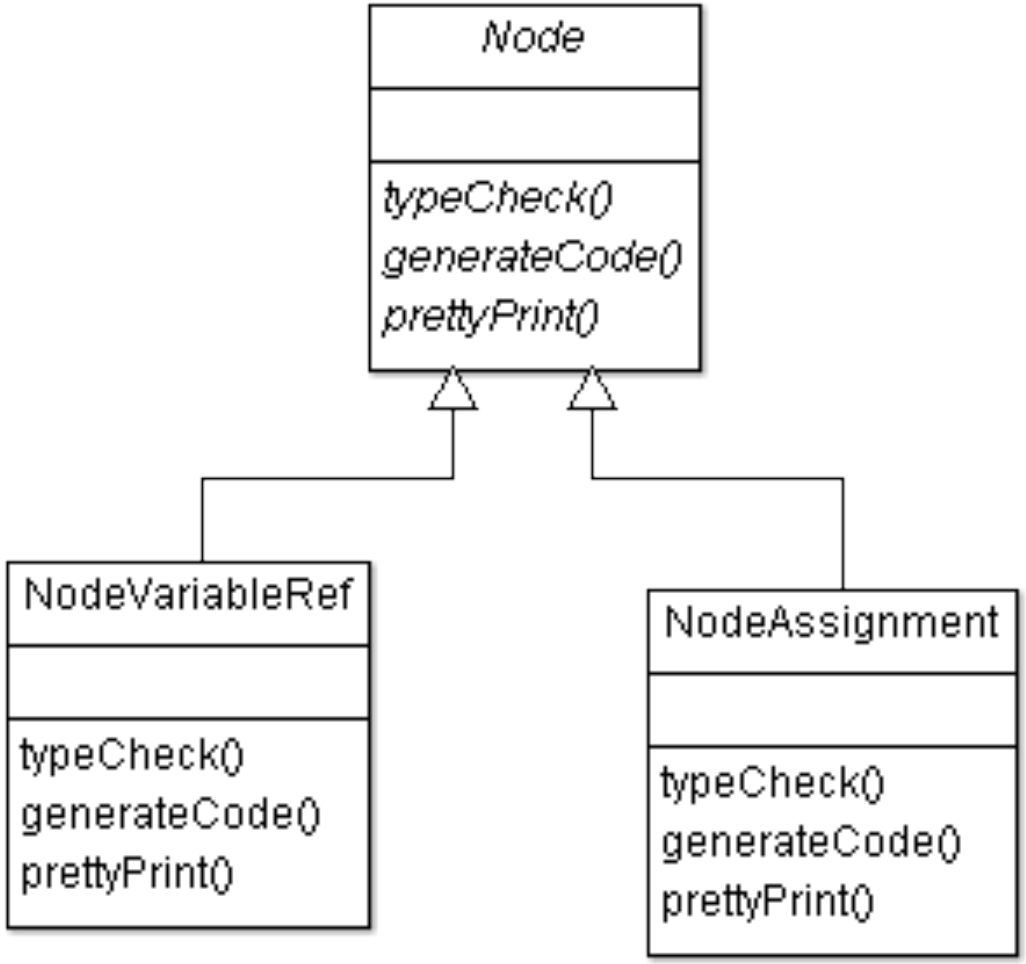
- reprezinta o operatie care se executa peste elementele unei structuri de obiecte
- permite sa definirea de noi operatii fara a schimba clasele elementelor peste care lucreaza

Motivatie

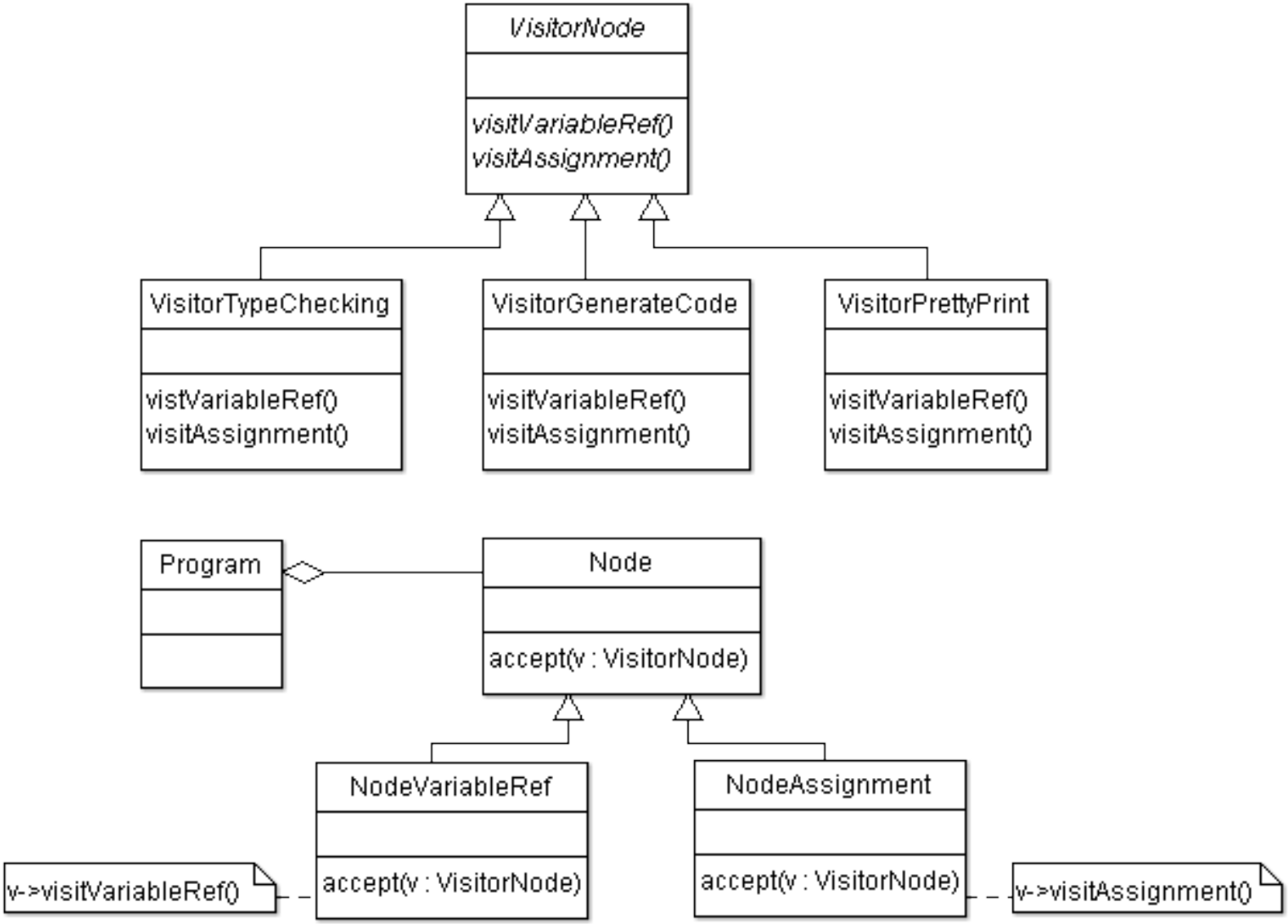
- Un compilator reprezinta un program ca un arbore sintactic abstract (AST). Acest arbore sintactic este utilizat atat pentru semantica statica (e.g., verificarea tipurilor) cat si pentru generarea de cod, optimizare de cod, afisare.
- Aceste operatii difera de la un tip de instructiune la altul. De exemplu, un nod ce reprezinta o atribuire difera de un nod ce reprezinta o expresie si in consecintele operatiile asupra lor vor fi diferite.
- Aceste operatii ar trebui sa se execute fara sa schimbe structura ASTului.
- Chiar daca structura ASTului difera de la un limbaj la altul, modurile in care se realizeaza operatiile sunt similare

Solutie necorespunzatoare

- “polueaza” structura de clase cu operatii care nu au legatura cu structura



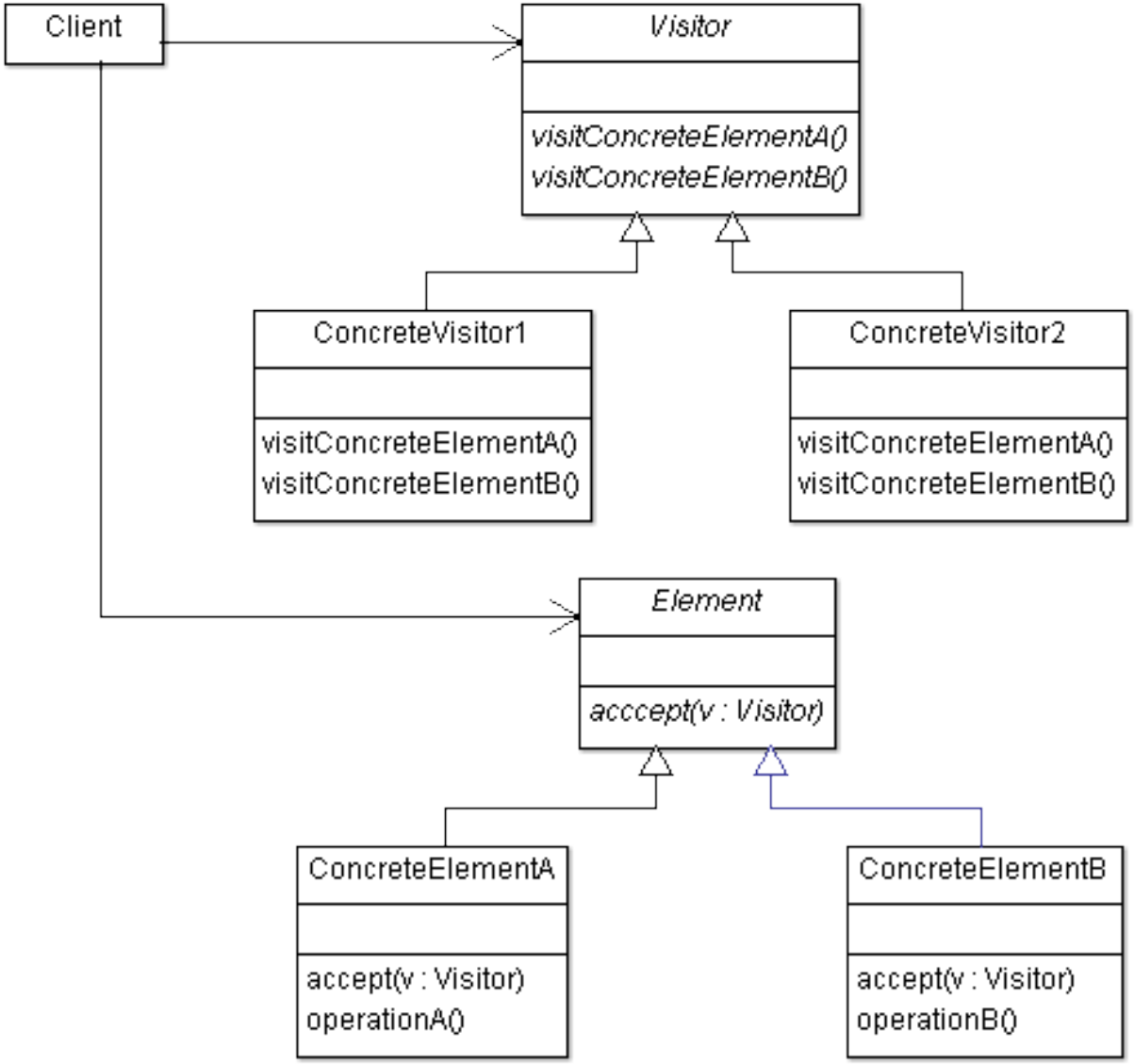
Solutia cu vizitatori



Aplicabilitate

- O structura de obiecte contine mai multe clase cu interfete diferite si se doreste realizarea unor operatii care depind de aceste clase
- Operatiile care se executa nu au legatura cu clasele din structura si se doreste evitarea “poluarii” acestor clase.
- Sablonul Visitor pune toate aceste operatii intr-o singura clasa.
- Cand structura este utilizata in mai multe aplicatii, in Visitor se pun exact acele operatii de care e nevoie.
- Clasele din structura se schimba foarte rar dar se doreste adaugarea de operatii noi peste structura.
- Schimbarea structurii necesita schimabrea interfetelor tuturor vizitatorilor.

Structura


Client
Visitor
visitConcreteElementA()
visitConcreteElementB()
ConcreteVisitor1
visitConcreteElementA()
visitConcreteElementB()
ConcreteVisitor2
visitConcreteElementA()
visitConcreteElementB()
Element
acccept(v : Visitor)
ConcreteElementA
accept(v : Visitor)
operationA()
ConcreteElementB
accept(v : Visitor)
operationB()

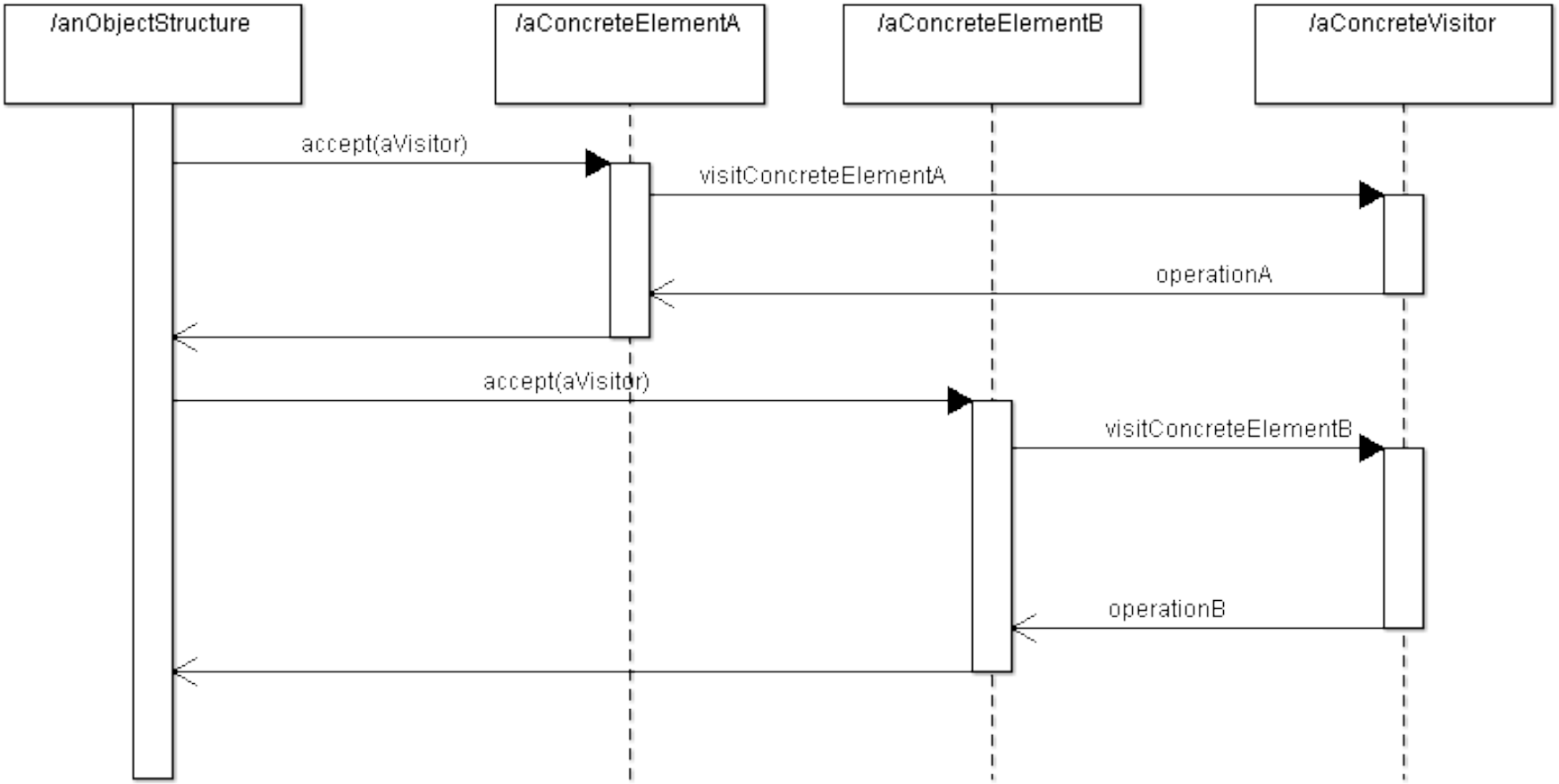
Participanti 1/2

- **Visitor** (NodeVisitor)
  - declara cate o operatie de vizitare pentru fiecare clasa ConcreteElement din structura. Numele operatiei si signatura identifica clasa care trimite cererea de vizitare catre vizitator. Aceasta permite vizitatorului sa identifice elementul concret pe care il viziteaza. Apoi, vizitatorul poate vizita elementul prin intermediul interfetei sale.
- **ConcreteVisitor** (TypeCheckingVisitor)
  - implementeaza fiecare operatie declarata de vizitator. Fiecare operatie implementeaza un fragment din algoritmul de vizitare care corespunde elementului din structura vizitat. Memoreaza starea algoritmului de vizitare, care de multe ori acumuleaza rezultatele obtinute in timpul vizitarii elementelor din structura.

Participanti 2/2

- **Element** (Node)
  - defineste operatii de acceptare, care au ca argument un vizitator
- **ConcreteElement** (AssignmentNode,VariableRefNode)
  - implementeaza operatia de acceptare
- **ObjectStructure** (Program)
  - poate enumera elementele sale
  - poate furniza o interfata la nivel inalt pentru un vizitator care viziteaza elementele sale
  - poate fi un “composite”

Colaborari (sequence diagram)



Consecinte 1/2

- *Visitor* face adaugarea de noi operatii usoara
- un vizitator aduna operatiile care au legatura intre ele si le separa pe cele care nu au legatura
- adaugarea de noi clase *ConcreteElement* la structura este dificila. Provoaca scimbarea interfetelor tutoror vizitatorilor. Cateodata o implementare implicita in clasa abstracta *Visitor* poate usura munca.
- spre deosebire de iteratori, un vizitator poate traversa mai multe ierarhii de clase
- permite calcularea de stari cumulative. Altfel, starea cumulativa trebuie transmisa ca parametru
- s-ar putea sa distruga incapsularea. Elementele concrete trebuie sa aiba o interfata puternica capabila sa ofere toate informatiile cerute de vizitator

Implementare 1/3

```
class Visitor {
public:
  virtual void visitElementA(ElementA*);
  virtual void visitElementB(ElementB*);
  // and so on for other concrete elements
protected:
  Visitor();
};
class Element {
public:
  virtual ~Element();
  virtual void accept(Visitor&) = 0;
protected:
   Element();
};
```

usual abstract methods

abstract class

abstract method

abstract class

Implementare 2/3

```
class ElementA : public Element {
public:
 ElementA();
 virtual void accept(Visitor& v) {
   v.visitElementA(this);
 }
};
class ElementB : public Element {
public:
  ElementB();
  virtual void accept(Visitor& v) {
   v.visitElementB(this);
  }
};
```

double dispatch

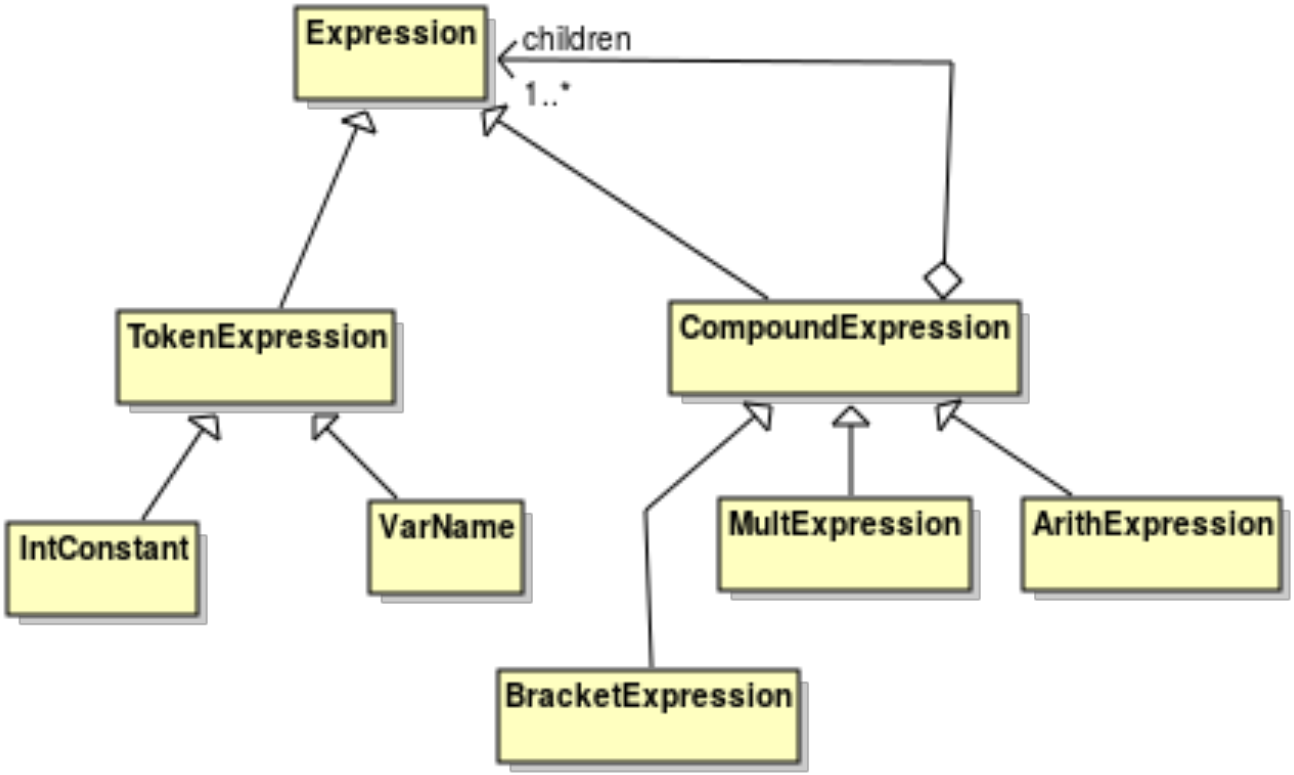
Implementare 3/3

- *Simple dispatch.* Operatia care realizeaza o cerere depinde de doua criterii: numele cererii si tipul receptorului. De exemplu, *generateCode* depinde de tipul nodului.
- *Double dispatch.* Operatia care realizeaza cererea depinde de tipurile a doi receptori. De exemplu, un apel *accept*() depinde atat de element cat si de vizitator.
- *Cine este responsabil de traversarea structurii de obiecte?*
  - structura de obiecte
  - vizitatorul
  - un iterator

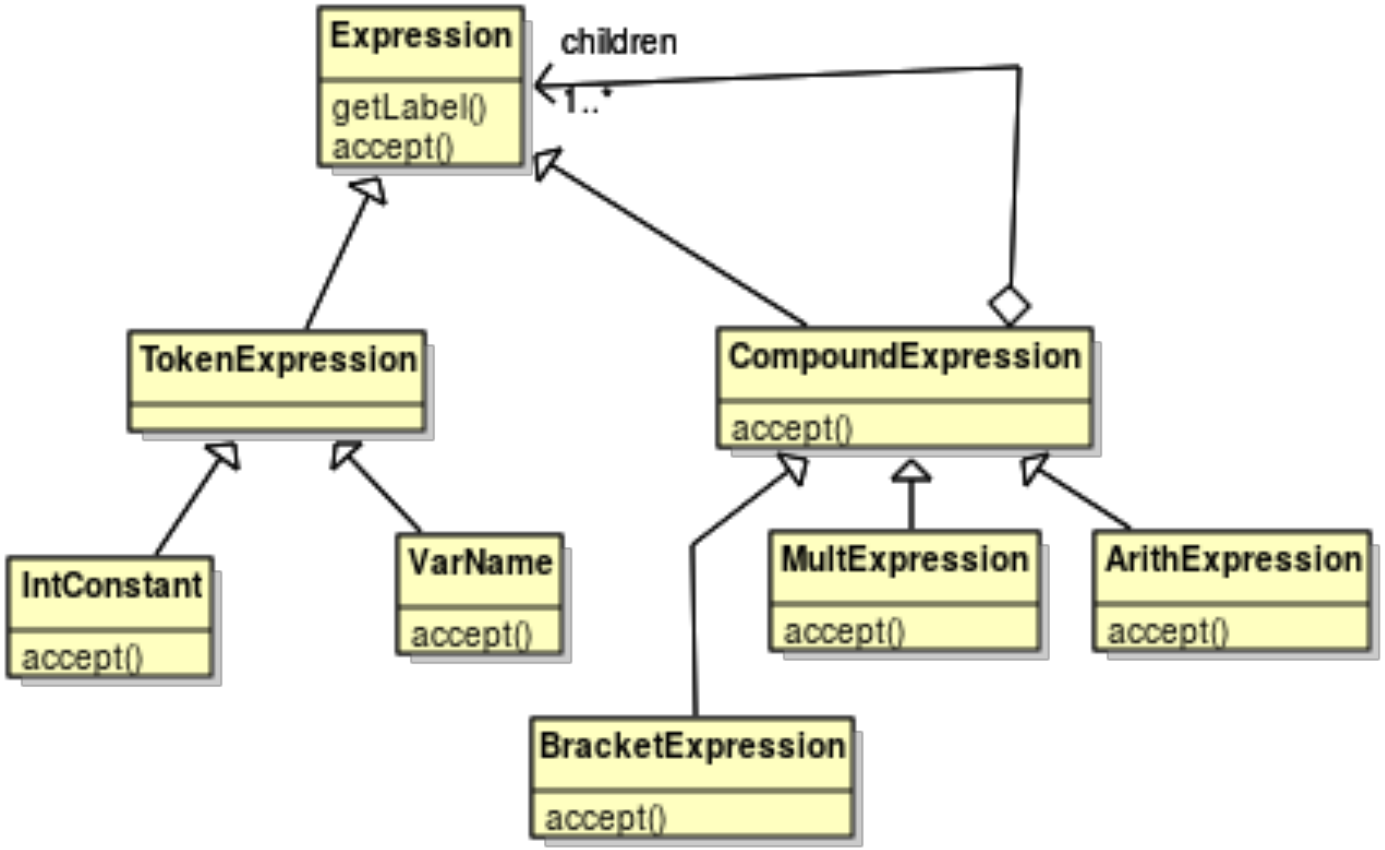
Aplicatie

- vizitatori pentru expresii
  - afisare
  - evaluare
-

Ierarhia Expression



Adaugarea metodei accept()



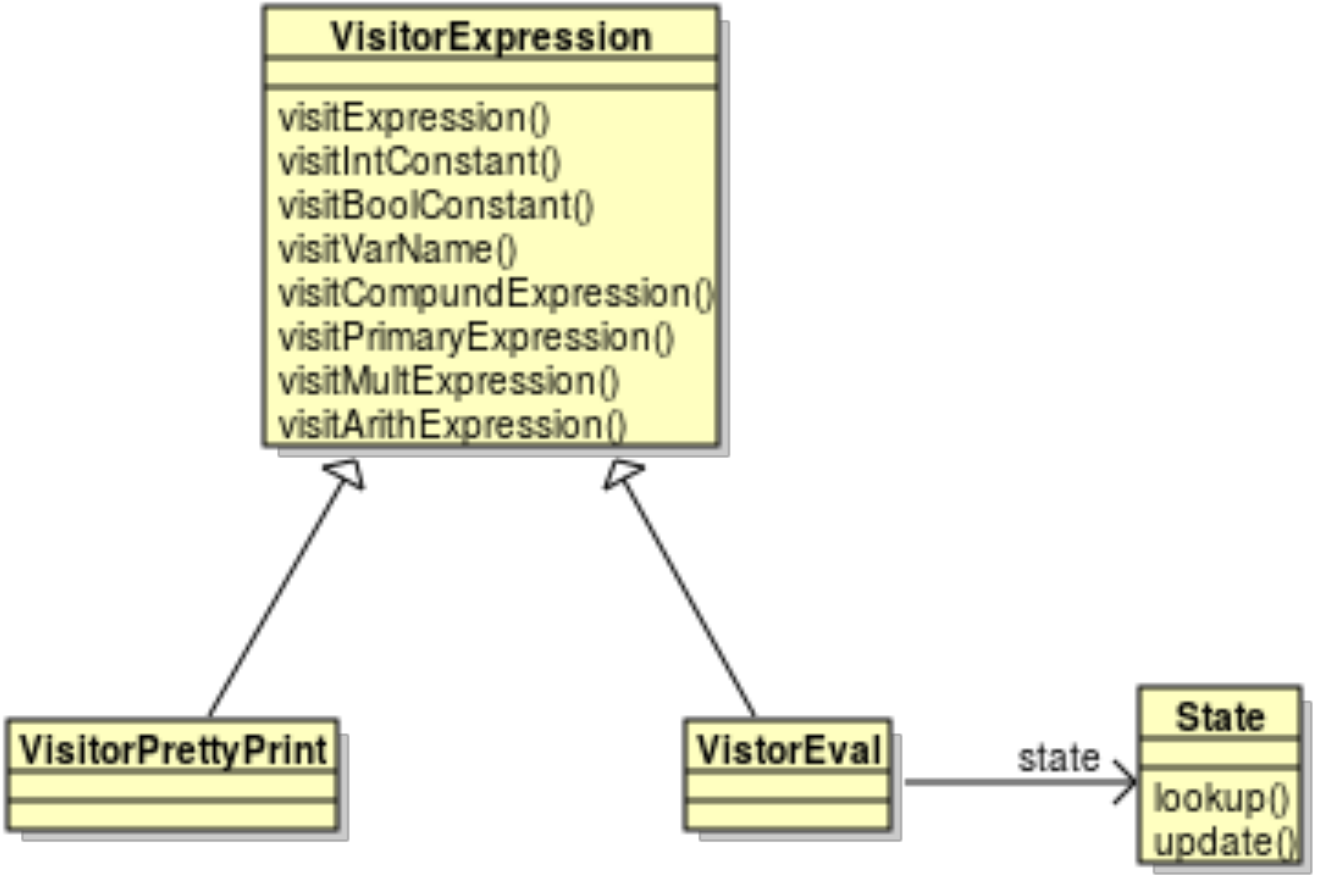
Adaugarea metodei accept() - C++

```
class Expression {
 …
 public: virtual void accept(Visitor& visitor) { }
};

class CompoundExpression : public Expression {
 …
 public: void accept(Visitor& visitor);
};

class ArithExpression : public CompoundExpression {
 public: void accept(Visitor& visitor);
};

class MultExpression : public CompoundExpression {
 public: void accept(Visitor& visitor);
};
…
```

Metoda accept()

```
void CompoundExpression::accept(Visitor& visitor) {
   list<Expression*>::iterator it;
   for ( it = children.begin(); it != children.end(); ++it)
     (*it)->accept(visitor);
  }

void ArithExpression::accept(Visitor& visitor) {
   this->CompoundExpression::accept(visitor);
   visitor.visitArithExpression(this);
  }

void MultExpression::accept(Visitor& visitor) {
   this->CompoundExpression::accept(visitor);
   visitor.visitMultExpression(this);
  }
…
```

Visitor pentru expresii



Visitor pentru expresii - C++

```
class Visitor {
 public:
  virtual void visitTokenExpression(TokenExpression*) {}
  virtual void visitIntConstant(IntConstant*) = 0;
  virtual void visitVarName(VarName*) = 0;
  virtual void visitCompoundExpression(CompoundExpression*) {}
  virtual void visitMultExpression(MultExpression*) {}
  virtual void visitArithExpression(ArithExpression*) {}
  virtual void visitBracketExpression(BracketExpression*) {}
 protected:
  Visitor(){};
};
```

Evaluarea

- reamintim ca vizitatorii pentru expresii compuse sunt apelati in post-ordine
- asta este OK pentru evaluare, unde subexpresiile trebui evaluate mai intai
- vom considera o stare cumulativa format dintr-o stiva
- fiecare sub-expresie va pune valoarea sa in stiva
- o (sub-)expresie compusa va gasi valorile sub-expresiilor sale in partea de sus a stivei
  - vizitarea va consta in adunare/inmultirea … acestor valori

Visitor pentru evaluarea de expresii

```
class VisitorEval : public Visitor {
public:
  union IntOrBool {
    int vint;
    bool vbool;
  };
  VisitorEval(State aState)
  {
    state = aState;
  }

  void visitIntConstant(IntConstant* constant);
  void visitVarName(VarName* variable);
  void visitMultExpression(MultExpression* me);
  void visitArithExpression(ArithExpression* ae);
  int getCumulateVal();
private:
  State state;
  stack<IntOrBool> valStack;
};
```

Visitor pentru evaluarea de expresii

```
void VisitorEval::visitIntConstant(IntConstant* constant) {
  IntOrBool ib;
  ib.vint = *constant;
  valStack.push(ib);
}


void VisitorEval::visitVarName(VarName* variable) {
  IntOrBool ib;
  ib.vint = state.lookup(*variable);
  valStack.push(ib);
}
```

Visitor pentru evaluarea de expresii

```
void VisitorEval::visitMultExpression(MultExpression* me) {
  list<string> label = me->getLabel();
  list<string>::reverse_iterator rits = label.rbegin();
  if (valStack.size() > 0) {
    int temp = valStack.top().vint;
    valStack.pop();
    while (rits != label.rend()) {
      if (*rits =="*") temp *= valStack.top().vint;
      if (*rits =="/") temp = valStack.top().vint / temp;
      if (*rits =="%") temp = valStack.top().vint % temp;
      valStack.pop();
      ++rits;
    }
    IntOrBool ib;
    ib.vint = temp;
    valStack.push(ib);
  }
}
```

explicatii pe tabla

Exercitii

- de adaugat expresii booleene
- de adaugat instructiuni (pe rand)
  - atribuirea
  - compunerea secventiala
  - if
  - while
  - declaratii de variabile
- de adaugat alti vizitatori
  - pretty printer
  - verificarea corectitudinii expresiilor
  - …

POO

Sablonul

Object Factory

Cuprins

- principiul inchis-deschis
- fabrica de obiecte (Abstract Object Factory)

(prezentare bazata pe GoF)

- studii de caz:
- *expression factory*

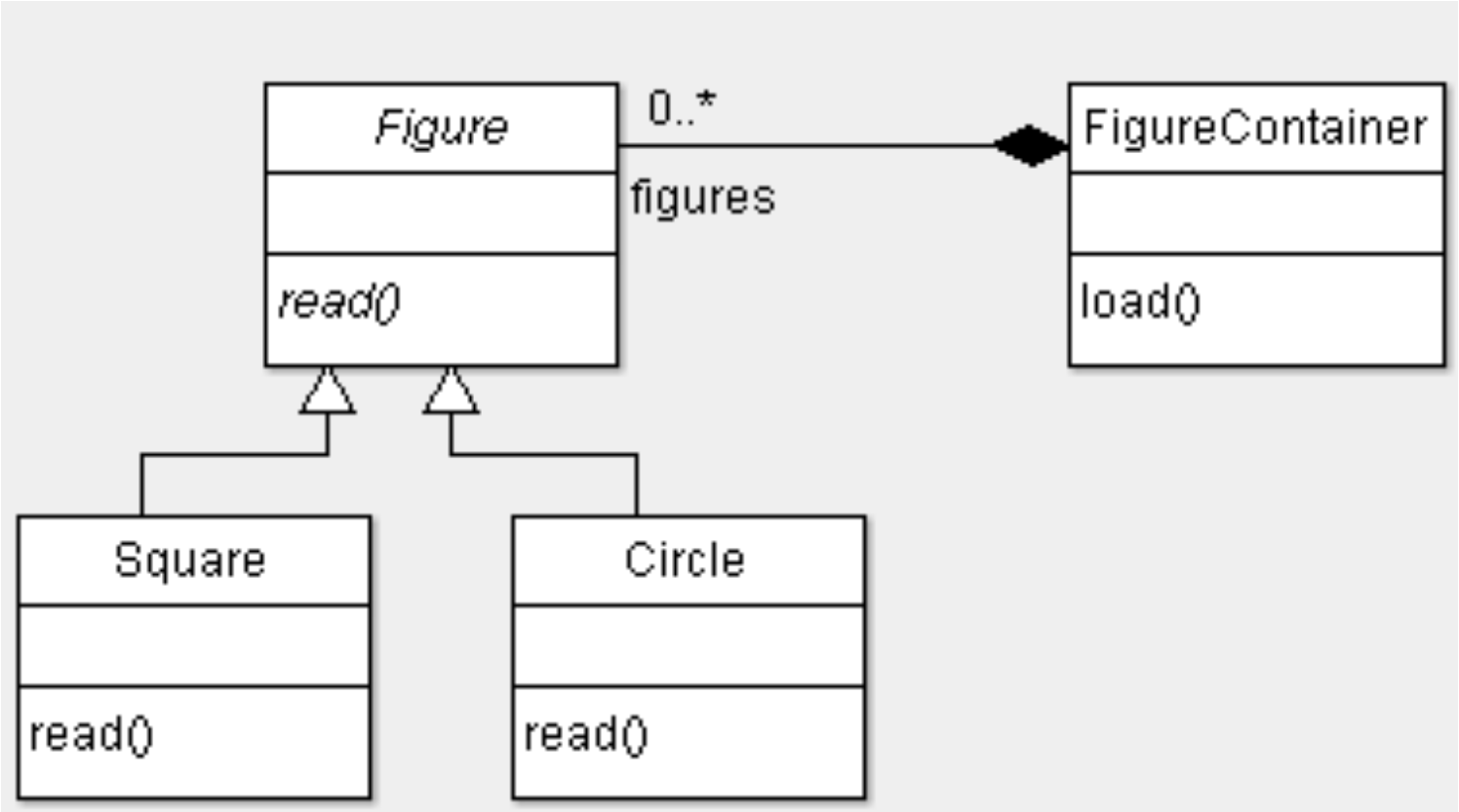
Principiul “inchis-deschis”

- “Entitatile software (module, clase, functii etc.) trebuie sa fie **deschise la extensii** si **inchise la modificare**” (Bertrand Meyer, 1988)

- “deschis la extensii” = comportarea modulului poate fi extinsa pentru a satisface noile cerinte

- “inchis la modificare” = nu este permisa modificarea codului sursa

Principiul “inchis-deschis” : exemplu



Principiul “inchis-deschis”: neconformare

```
void FigureContainer::load(std::ifstream& inp)
{
    while (inp)
    {
        int tag;
        Figura* pfig;
        inp >> tag;
        switch (tag)
        {
          case SQUAREID:
            ...
          case CIRCLEID:
            ...
        }
    }
}
```

eticheta figura

citeste tipul figurii ce urmeaza a fi incarcate

adaugarea unui nou tip de figura presupune modificarea acestui cod

pfig = new Square;
pfig.read(inp);

Square::read()

pfig = new Circle;
pfig.read(inp);

Circle::read()

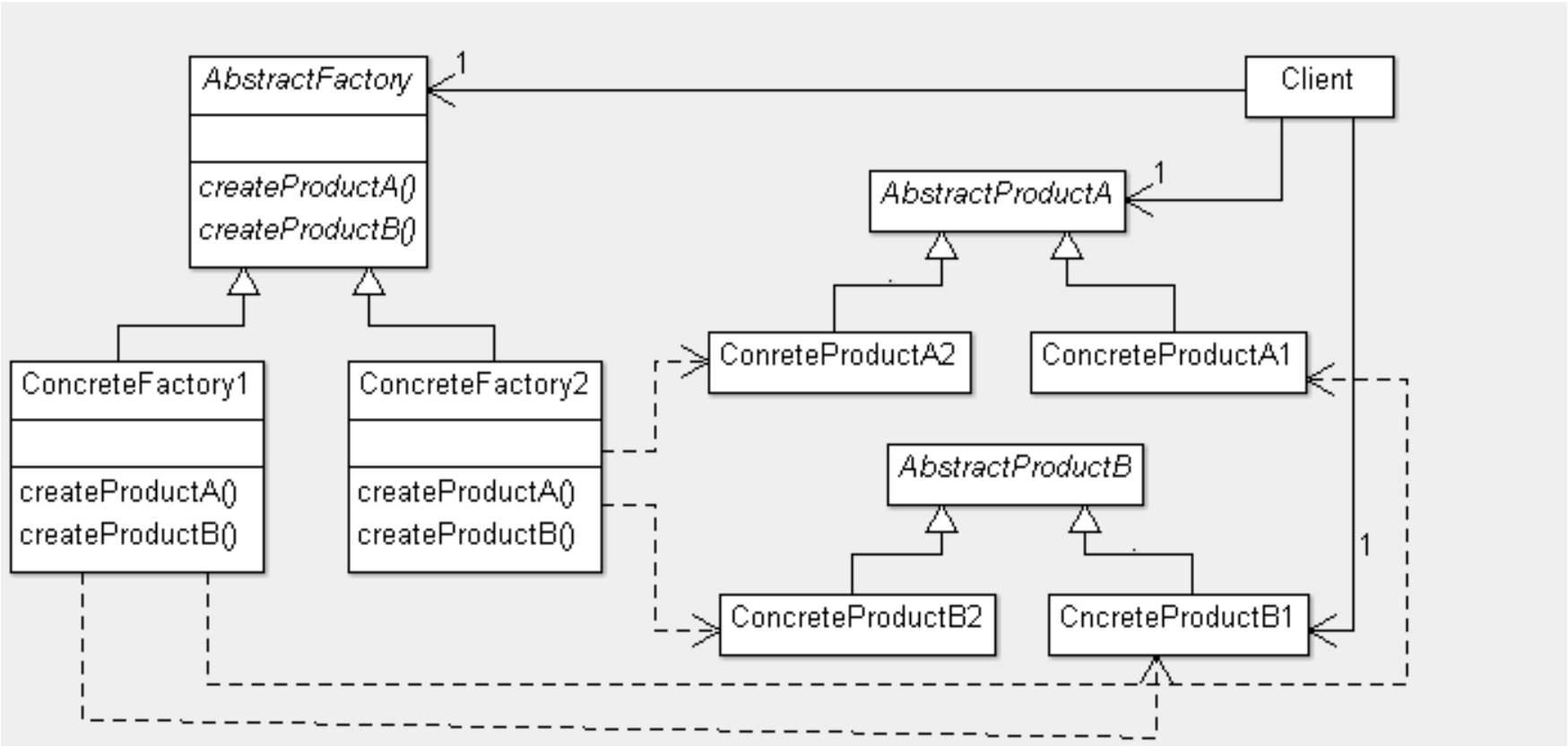
Principiul “inchis-deschis”

O posibila solutie: fabrica de obiecte

Fabrica de obiecte (Abstract Factory)

- intentie
  - de a furniza o interfata pentru crearea unei familii de obiecte intercorelate sau dependente fara a specifica clasa lor concreta
- aplicabilitate
  - un sistem ar trebui sa fie independent de modul in care sunt create produsele, compuse sau reprezentate
  - un sistem ar urma sa fie configurat cu familii multiple de produse
  - o familie de obiecte intercorelate este proiectata astfel ca obiectele sa fie utilizate impreuna
  - vrei sa furnizezi o biblioteca de produse si vrei sa fie accesibila numai interfata, nu si implementarea

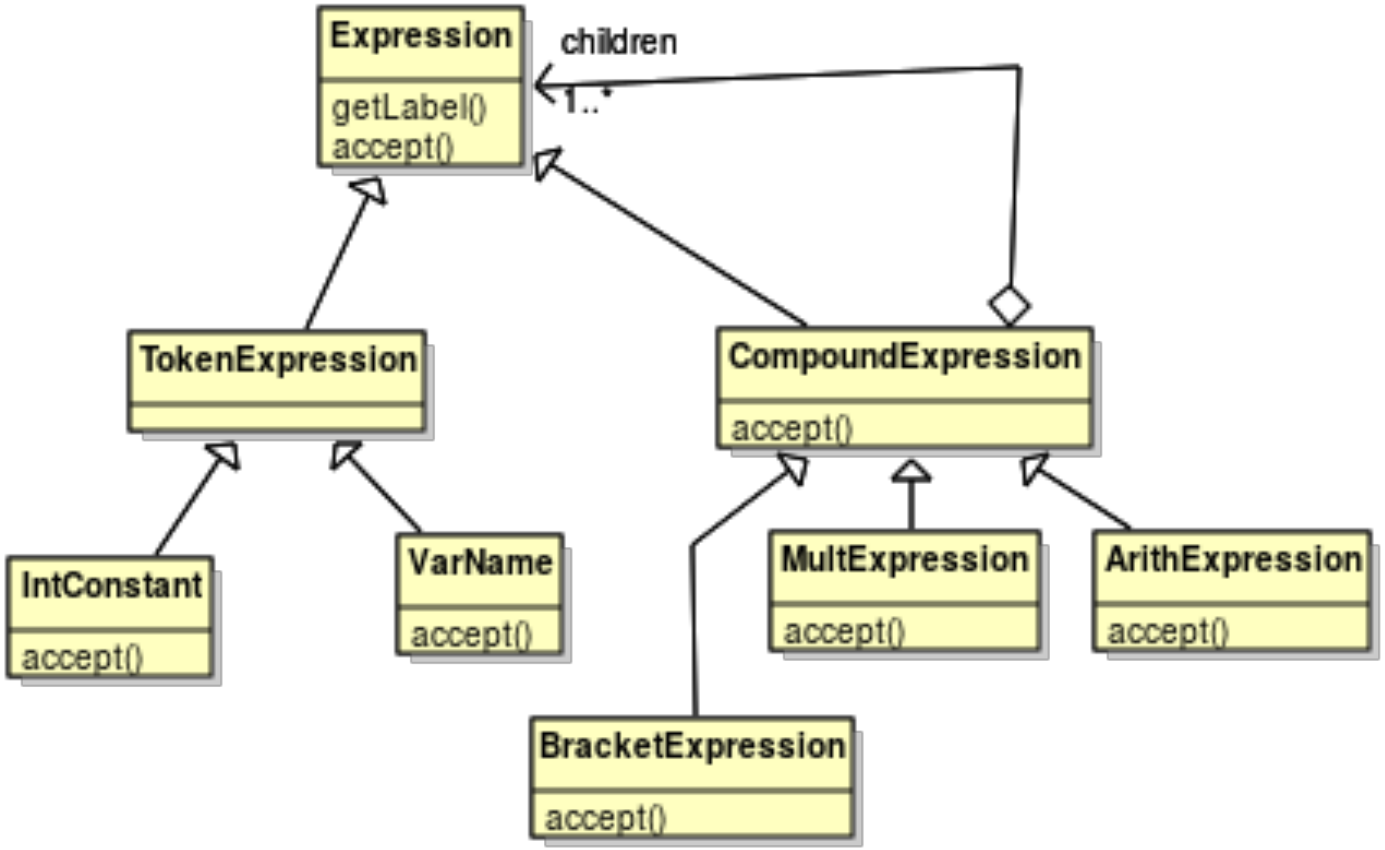
Fabrica de obiecte:: structura



Fabrica de obiecte

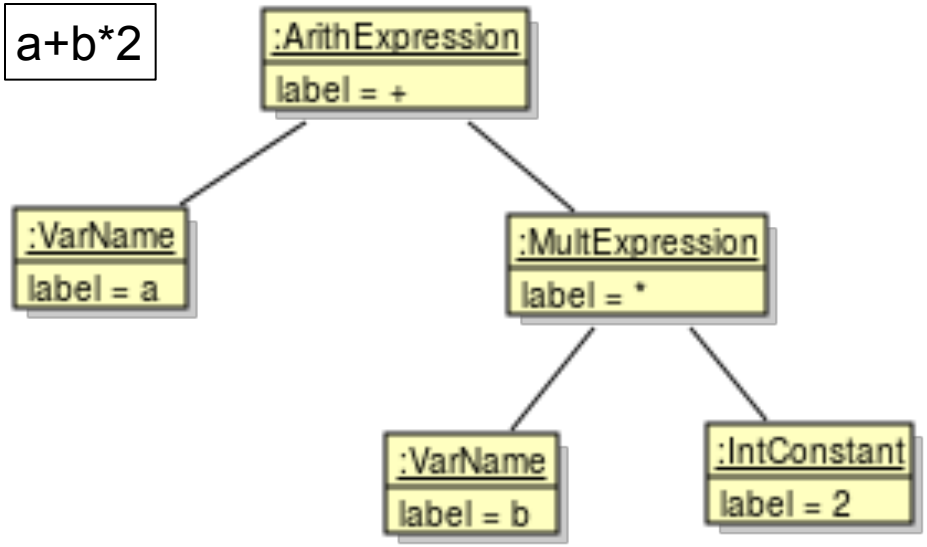
- colaborari
  - normal se creeaza o singura instanta
- consecinte
  - izoleaza clasele concrete
  - simplifica schimbul familiei de produse
  - promoveaza consistenta printre produse
  - suporta noi familii de produse usor
  - respecta principiul deschis/inchis
- implementare
  - se face pe baza studiului de caz “expression factory”

Expresii : : structura



Expressions: Problema (diagr. de obiecte)

a+b*2



serializare

```
<arith>
  <label> [+] </label>
  <varName> a </varName>
  <mult>
    <label> [*] </label>
    <varName> b </varName>
    <intConstant> 2 </
   intConstant>
  </mult>
</arith>
```

deserializare

notatie XML

Expressions: Problema

- serializare
  - vizitator
- deserializare

```
switch (tag)
    {
        case <sum>:
            ...
        case <prod>:
            ...
        case <constant>:
            ...
        case <variable>:
            ...
    }
```

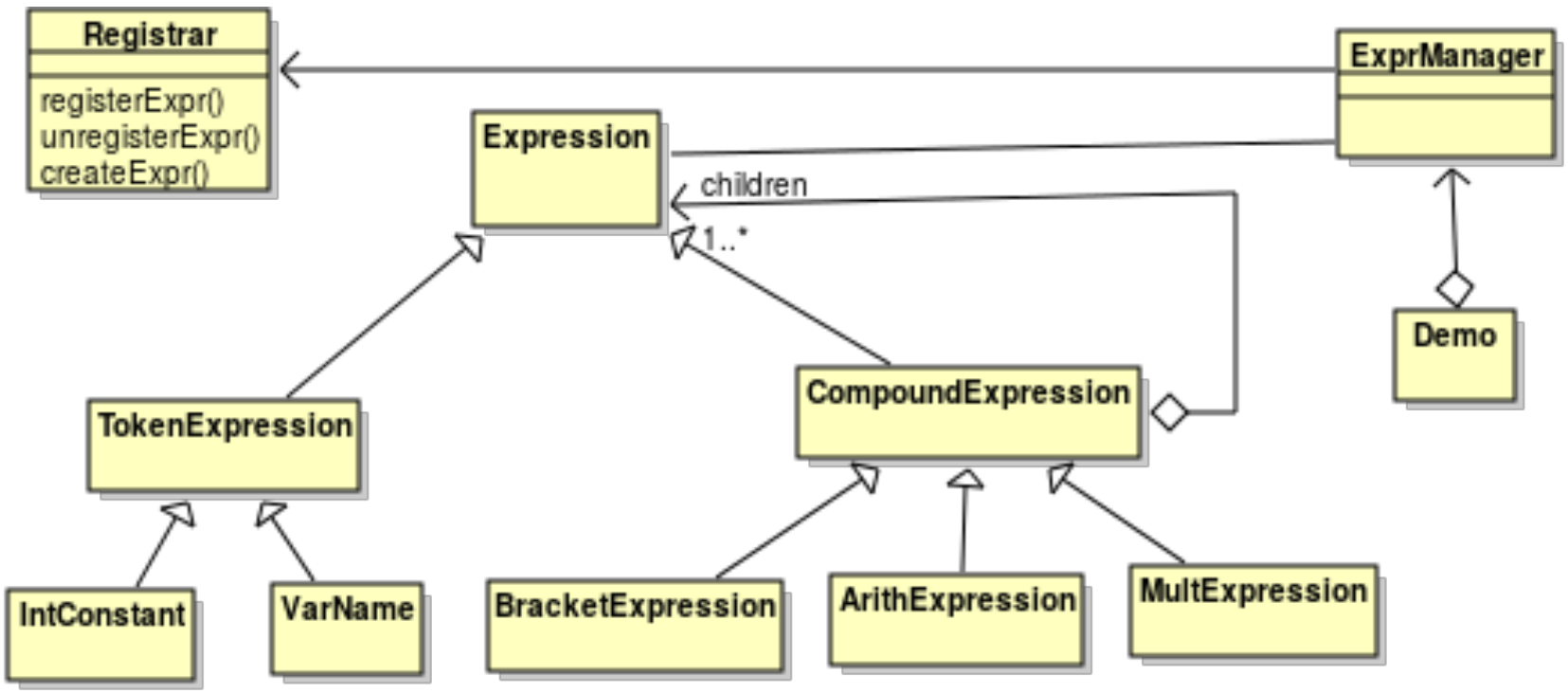
se incalca principiul inchis-deschis

Solutia: object factory

- corespondenta cu modelul standard
  - AbstractProductA = Expression
  - AbstractProductB = Statements (neimplementat inca, lasat ca exercitiu)

  - ConcreteFactory = Registrar (registru de expresii, instructiuni)
  - Client = ExprManager (responsabila cu deserializarea)

“Object factory” pentru expresii



Registru de clase (Registrar)

- este o clasa care sa gestioneze tipurile de expresii
  - inregistreaza un nou tip de expresie (apelata ori de cate ori se defineste o noua clasa derivata)
  - eliminarea unui tip de expresie inregistrat (stergerea unei clase derivate)
  - crearea de obiecte expresie
    - la nivel de implementare utilizam perechi

      (tag, createExprFn)
    - ... si functii delegat (vezi slide-ul urmator)
- se poate utiliza sablonul Singleton pentru a avea o singura fabrica (registru)

Functii delegat (callback)

- o functie **delegat** (**callback)** este o functie care nu este invocata explicit de programator; responsabilitatea apelarii este delegata altei functii care primeste ca parametru adresa functiei delegat
- Fabrica de obiecte utilizeaza functii delegat pentru crearea de obiecte**:** pentru fiecare tip este delegata functia carea creeaza obiecte de acel tip

- pentru “expression factory” declaram un alias pentru tipul functiilor de creare a obiectelor Expression

```
typedef Expression* ( *CreateExprFn )();
```

Registrar 1/3

```
class Registrar
{
  bool registerExpr(string tag,
                    CreateExprFn createExprFn )
  {
      return catalog.insert(
         std::pair<string, CreateExprFn>(
                        tag, createExprFn)
                              ).second;
  }
```

metoda responsabila cu inregistrarea unui nou tip de obiecte Expression

inserarea in catalog (un map)

a doua componenta a valorii intoarse de insert (inserare cu succes sau fara succes)

Registrar 2/3

```
void unregisterExpr(string tag)
   {
       catalog.erase(tag);
   }

Expression* createExpr(string tag)
   {
       map<string, CreateExprFn>::iterator i;
       i = catalog.find(tag);
       if ( i == catalog.end() )
             throw string("Unknown expression tag”);
       return (i->second)();
   }
```

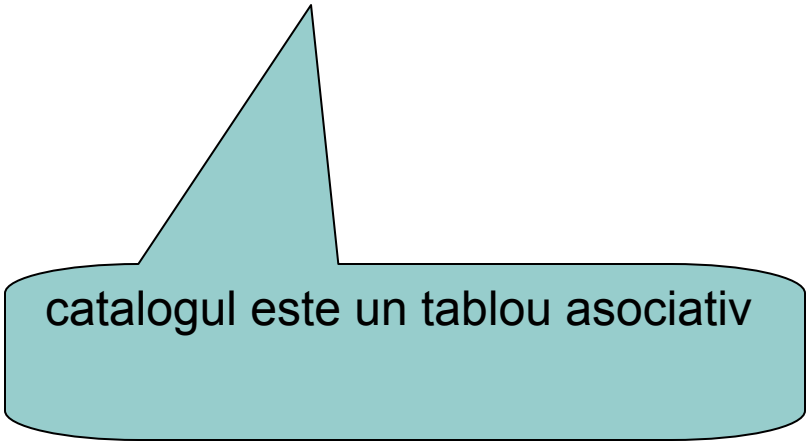
metoda responsabila cu eliminarea unui obiect tip Expression

metoda responsabila cu crearea de obiecte Expression

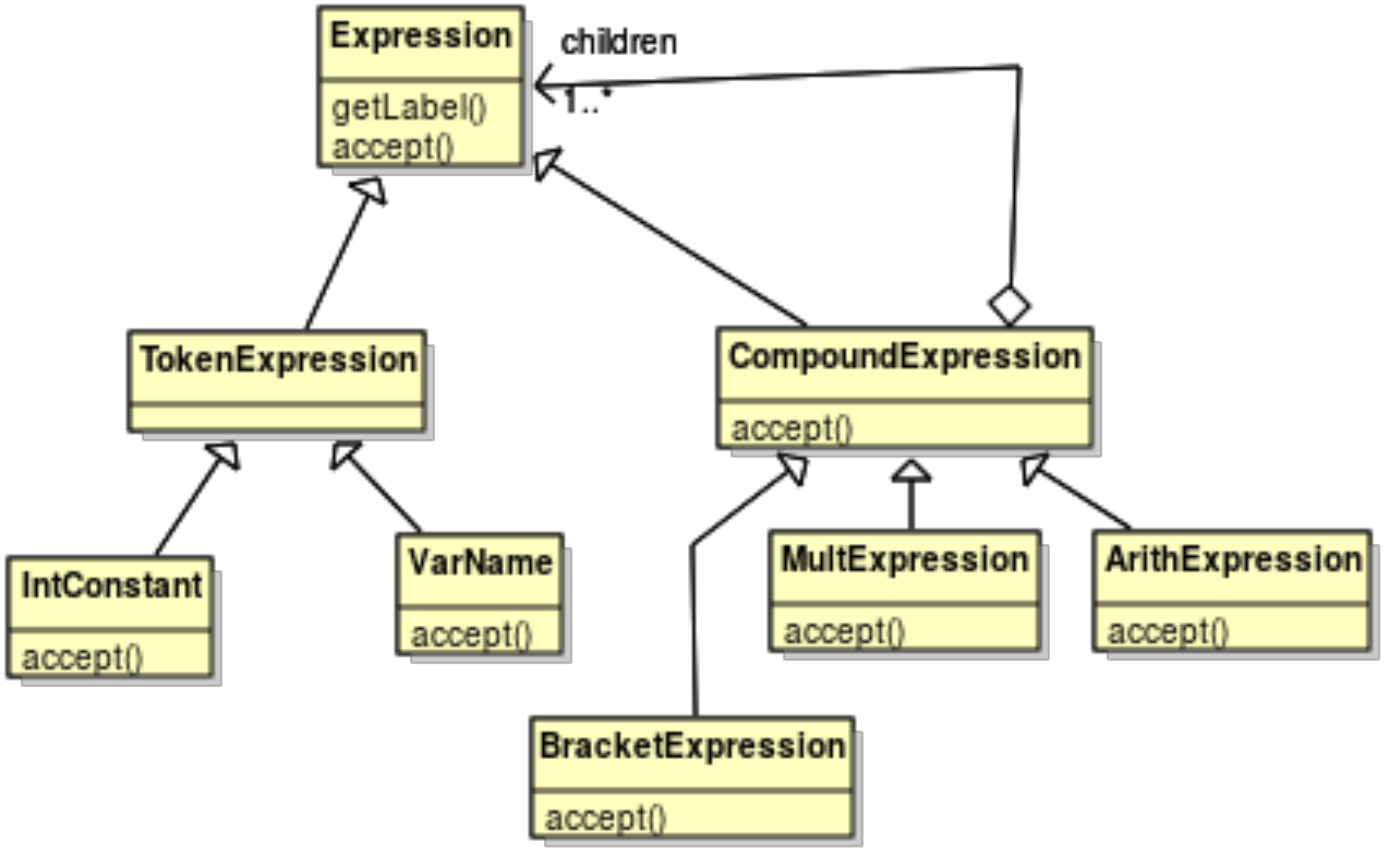
de fapt deleaga aceasta responsabilitate metodei care corespunde tipului dat ca parametru

Registrar 3/3

```
protected:

  map<string, CreateExprFn> catalog;

};
```



Produsele din familia Expression



expr-manager.h – functii de creare obiecte

```
Expression* createIntConstant() {
      return new IntConstant();
}

Expression* createVarName() {
      return new VarName();
}

Expression* createMultExpression() {
      return new MultExpression();
}
…
```

Clasa ExprManager - constructorul

```
class ExprManager {
public:
  ExprManager() {
    reg = new Registrar();
    reg->registerExpr("<intConstant>",
                      createIntConstant);
    reg->registerExpr("<varName>",
                      createVarName);
    reg->registerExpr("<mult>",
                      createMultExpression);
    reg->registerExpr("<arith>",
                      createArithExpression);
  }
```

Deserializarea (fabrica de obiecte din descrieri XML)

```
public:
  Expression* loadf(ifstream& f)
  {
      if (f.eof())
            throw "Unknown file.";
      string tag;
      f >> tag;
      return loadfRec(f, tag);
  }
protected:
  Registrar *reg;
```

Functia recursiva de creare obiecte 1/3

```
protected:
Expression* loadfRec(ifstream& f,
                     string tag)
{
  Expression* expr1 = reg->createExpr(tag);
  string endTag = tag.insert(1,"/");
  Expression* expr2;
```

creaza obiectul pentru nodul curent

calculeaza tagul de sfarsit

memoreaza expresiile componente, daca expr1 este compusa

... cazul obiectelor compuse

```
if (expr1->getCompoundExpression()) {
  if (f.eof())throw "File illformatted.";
  string labelInfo;
  f >> labelInfo;
  f >> labelInfo;
  expr1->setLabel(toLabel(labelInfo));
  f >> labelInfo;
  string nextTag;
  f >> nextTag;
  while (endTag != nextTag && !f.eof()) {
    expr2 = loadfRec(f, reg, nextTag);
    expr1->addChild(expr2);
    f >> nextTag;
  }
}
else
```

citeste eticheta nodului compus

citeste urmatorul tag

daca nu s-a ajuns la tagul de sfarsit, inseamna ca avem o noua componenta pe

... cazul obiectelor elementare

```
   else {
      if (f.eof())
            throw "File illformatted.";
      expr1->loadInfo(f);
      if (f.eof())
            throw "File illformatted.";
      f >> tag;
   }
   return expr1;
 }
};
```

incarca informatia din nodul frunza

consuma tagul de sfarsit

Concluzii

- sabloanele de proiectare constituie o modalitate de a învăța cum să vă construiesc programele.
- un sablon de proiectare este un sfat care vine de la oameni care au distilat soluțiile lor cele mai comune în sfaturi de cunoștințe simple, digerabile si cu nume sugestiv
- POO și sabloanele de proiectare sunt subiecte distincte
- POO te învață cum să programezi, este o metodologie de programare sau un concept de programare
- sabloanele de proiectare te învață cum să te gândești despre programe, iti sugerează metode de construire a unor clase / obiecte pentru a rezolva un anumit scenariu într-un program, metode dovedite a avea succes
- multe alte sabloane utile se gasesc în GoF
-